Prepare answers before the exercise class. In class you will present them.

1.1 Which group of properties is characteristic for any distributed system?

T F (T=true/F=false)

☐ ☐ Independent computers, connected by a communication network, synchronized clocks

☐ ☐ Independent computers, common shared memory, asynchronous communication network

☐ ☐ Messages-based communication, common task, common shared memory

☐ ☐ Multiple independent computers, no common clock, no shared memory

Sample solution.

Comment: F/F/F/T

- *Usually there are no synchronized clocks. This is one of the biggest challenges that algorithms for distributed systems are facing.*
- *We assume that nodes interact only by message passing, not via shared memory.*

1.2 Which attributes are essential characteristics of a dependable distributed system?

T F (T=true/F=false)

☐ ☐ Reliability, Maintainability, Availability

☐ ☐ Portability, Scalability, Reliability

☐ ☐ Safety, Security, Integrity

☐ ☐ Timeliness, Availability, Cost-effectiveness

Sample solution.

Comment: Reliability, maintainability, and availability are without doubt essential characteristics. Portability and scalability sometimes can be useful properties, but certainly are not universally characteristic for all dependable systems. You can argue that safety, security, and integrity are essential. Timeliness and cost-effectiveness are sometimes desired, but certainly not always essential characteristics.

1.3 Dependability

a) In two or three sentences: dependability is not the same thing as reliability. What is the difference?

b) Name the three dependability attributes discussed with the measures MTTF, MTBF, and MTTR, and say which measure belongs to which attribute.

c) System A fails once per year and takes one day to repair. System B fails once per week and takes one hour to repair. Using Availability $= \mathrm{MTBF}/(\mathrm{MTBF} + \mathrm{MTTR})$, which system is more available? State the MTBF and MTTR you use, and the two availability values.

Sample solution.

a) Dependability, in Laprie's definition, is the quality of a delivered service such that reliance can justifiably be placed on that service. Reliability is one attribute of that: continuous delivery of correct service. A system can be dependable because it is available, safe, or maintainable, not only because it rarely fails.

b) Reliability is measured by MTTF or MTBF (how long correct service continues). Maintainability is measured by MTTR (how long restoration takes). Availability is $\mathrm{MTBF}/(\mathrm{MTBF}+\mathrm{MTTR})$*, the fraction of time in which correct service is delivered.*

c) For A, take $\mathrm{MTBF} = 364$ *days and* $\mathrm{MTTR} = 1$ *day:* $364/365 \approx 0.9973$*. For B, take* $\mathrm{MTBF} = 167$ *hours and* $\mathrm{MTTR} = 1$ *hour:* $167/168 \approx 0.9940$*. A is more available. B fails much more often, and the shorter repair does not make up for that.*

1.4 Errors, failures, and faults

a) Put error, failure, and fault in causal order, and give a one-sentence definition of each.

b) A program in C uses a local variable that the programmer never initialized. The function is later called, and the program crashes. Identify the error, the failure, and the fault.

c) Give a second example of your own, from hardware or from a service you use. The example on the lecture slides (a stuck RAM bit) does not count.

Sample solution.

a) Causal order: fault, then error, then failure. A fault is the adjudged cause. An error is the part of the system state that can lead to a failure. A failure is a delivered service that deviates from the system function.

b) Fault: the missing initialization. Error: after the call, the variable holds an unintended value, so the internal state is wrong. Failure: the crash, which is visible at the service boundary. The fault can sit in the code for a long time without an error, until that function runs.

c) One acceptable example: a disk in a RAID array fails (fault), the stripe that lived on that disk is missing (error), and a read still succeeds because the parity reconstructs it. There is no failure yet. A failure appears only when a second disk fails and the data can no longer be reconstructed.

1.5 How to get there

The lecture distinguishes fault prevention, fault removal, fault forecasting, and fault tolerance. Assign each measure to one of the four. One sentence of justification is enough.

a) Code review and testing before release.

b) Storing the same data on three disks and reading a majority.

c) Estimating how often a disk model fails, from field data.

d) Choosing a language in which an uninitialized variable does not compile.

Sample solution.

a) Fault removal: the fault is already in the system, and testing is there to find it before users do. b) Fault tolerance: a faulty disk is masked by the other two, so the service stays correct. c) Fault forecasting: the goal is to estimate how many faults to expect, not to prevent or hide a particular one. d) Fault prevention: the fault is avoided by construction, instead of being detected later.

1.6 Problems in Distributed Systems

a) In your opinion, what are the most important practical problems and challenges in distributed systems? You should especially focus on *technical problems* that should be solved by *algorithms*! Consider the experience you have from other lectures and projects, as well as outside of the university.

b) Which of the problems are especially interesting for you? For which of them do you want to learn more in this lecture?

Sample solution.

(no sample solution; discussion in class)